Benchmarking exact hypervolume computation: Chan's Section-2 and Section-4.2 algorithms in Python and C

Michael Emmerich
University of Jyväskylä, Finland

August 21, 2026

Abstract

We compare implementation variants of exact hypervolume computation across six objective-space dimensions and two families of point sets: the Python reference implementations of Chan's Section-2 divide and conquer and his Section-4.2 orthant algorithm, a C99 port of the Section-2 algorithm at two base-case settings, and three classical baselines — a dimension sweep after Guerreiro, Fonseca and Emmerich (CCCG 2012), the WFG algorithm of While, Bradstreet and Barone (IEEE TEC 2012), and a vendored Yıldız–Suri-style anchored solver. All variants are run on byte-identical input and agree on the computed hypervolume to within 1.1e-14 relative, which is the primary correctness evidence reported here. The C port of Chan's algorithm is one to two orders of magnitude faster than the Python original; among the Python variants, the dimension sweep is the fastest on every measured case, while the partitioning-based methods' worst-case guarantees come with visibly larger constants.

1 Setup

Machine. Windows-11-10.0.26200-SP0, Python 3.13.15. Times are wall clock, taken in a fresh subprocess per measurement; fast cases are repeated until the total exceeds 50 ms and the per-call mean is reported. Any measurement exceeding 60.0 s is recorded as a timeout (t/o), and that variant is not retried at a larger n for the same dimension (the curves are monotone in n), which is shown as –.

Reference point. All objectives are minimised against the reference point $(1.2, \ldots, 1.2)$. Datasets are generated from a fixed seed (20260821), written to disk, and read back by every variant, so all implementations see exactly the same floating-point input.

1.1 Variants

Py §2 `chan_hypervolume.hypervolume`, the Section-2 divide and conquer, with the $O(n^2 d)$ dominance prefilter.

Py §2 np The same, with `prefilter=False`. On these datasets every point is already non-dominated, so the filter can only cost time, never save it.

Py §4.2 `chan_orthant_dby3.hypervolume_dby3`, the Section-4.2 orthant algorithm with $\tilde{F}$ compression on.

Py §4.2 nc The same with `use_compression=False`, isolating what the compression step buys.

C §2 The C99 port of Section 2 in `c/`, base case $b = 2$, matching the Python default.

C §2 b=8 The same C code with base case $b = 8$: the recursion stops earlier and finishes by inclusion–exclusion.

Py DS `hv_baselines.hypervolume_ds`: dimension sweep with the classical $O(n \log n)$ 3-D base case, organised after Guerreiro, Fonseca, and Emmerich (CCCG 2012) in higher dimensions (exclusive contributions recomputed per point).

Py WFG `hv_baselines.hypervolume_wfg`: the WFG algorithm of While, Bradstreet, and Barone (IEEE TEC 16(1), 2012), recursive exclusive hypervolumes over dominance-pruned limit sets.

Py YS The vendored Yıldız–Suri-style anchored solver from `benchmarks/vendor/`, applied through the transform $q = r - y$ (Yıldız and Suri, SoCG 2012).

1.2 Datasets

spherical. n points drawn uniformly in the positive orthant and normalised to the unit sphere. Every point is mutually non-dominated in all d objectives. This is the standard hard case: dominance pruning removes nothing, so the full recursion is exercised.

cliff. The first two coordinates lie on a quarter-circle arc, and the remaining $d-2$ coordinates are uniform noise. Because the first two coordinates alone are already mutually non-dominated, no point can dominate another regardless of the other objectives: the entire dominance structure sits in a two-dimensional slice, while the remaining $d - 2$ objectives contribute geometry but no order. The front is as large as in the spherical case, but degenerate in a way the recursion must discover for itself. It is the more adversarial of the two for any method that hopes to exploit dominance.

2 Results

2.1 Timings

Table 1 and Table 2 give wall-clock seconds per call.

d	n	Py §2	Py §2 np	Py §4.2	Py §4.2 nc	Py DS	Py WFG	Py YS	C §2	C §2 b=8
3	100	0.0158	0.0131	0.0319	0.0312	0.0027	0.0195	0.0012	0.00068	0.00077
3	200	0.0401	0.0306	0.0761	0.0755	0.0102	0.0725	0.0025	0.0014	0.0015
4	60	0.0300	0.0291	0.1661	0.1678	0.0075	0.0314	0.0117	0.0014	0.0011
4	120	0.0920	0.0884	0.4070	0.4122	0.0338	0.1079	0.0329	0.0043	0.0041
5	40	0.0600	0.0592	0.4023	0.4081	0.0135	0.0509	0.1013	0.0021	0.0011
5	80	0.2320	0.2286	1.84	1.70	0.0607	0.3741	0.4587	0.0071	0.0045
7	25	0.1935	0.2056	1.35	1.33	0.0297	0.1133	1.04	0.0087	0.0018
7	50	1.54	1.57	11.3	11.4	0.2655	2.40	11.5	0.0510	0.0173
9	20	0.8681	0.8689	4.93	4.76	0.0708	0.2983	17.5	0.0315	0.0055
9	40	12.4	11.9	*t/o*	*t/o*	1.30	38.7	*t/o*	0.3940	0.0730
10	20	2.31	2.37	11.1	10.8	0.1366	0.3574	*t/o*	0.0750	0.0085
10	40	33.8	33.2	*t/o*	*t/o*	2.40	25.0	–	1.22	0.1990

Table 1: spherical: seconds per call.

d	n	Py §2	Py §2 np	Py §4.2	Py §4.2 nc	Py DS	Py WFG	Py YS	C §2	C §2 b=8
3	100	0.0135	0.0114	0.0276	0.0293	0.0026	0.0341	0.0014	0.00028	0.00024
3	200	0.0354	0.0271	0.0593	0.0598	0.0102	0.1238	0.0031	0.0011	0.00093
4	60	0.0162	0.0160	0.0804	0.0823	0.0067	0.0311	0.0090	0.00046	0.00035
4	120	0.0518	0.0480	0.2065	0.2163	0.0255	0.1356	0.0225	0.0014	0.0013
5	40	0.0307	0.0294	0.2290	0.2025	0.0053	0.0486	0.0828	0.0011	0.00057
5	80	0.0915	0.0881	0.5131	0.5112	0.0194	0.2360	0.3169	0.0028	0.0020
7	25	0.1213	0.1290	0.8590	0.8570	0.0130	0.1263	1.01	0.0048	0.0014
7	50	0.5259	0.5288	4.11	4.17	0.0530	1.44	10.1	0.0193	0.0056
9	20	0.4616	0.4460	3.13	3.05	0.0220	0.0550	17.1	0.0155	0.0023
9	40	2.78	2.76	27.6	27.9	0.1964	4.03	*t/o*	0.1030	0.0168
10	20	0.9444	0.9505	5.93	5.69	0.0356	0.1202	*t/o*	0.0285	0.0036
10	40	9.39	9.34	*t/o*	*t/o*	0.3723	19.3	–	0.3210	0.0395

Table 2: cliff: seconds per call.

2.2 Speedup of the C port

Table 3 divides the fastest Python variant for each case by the C time, so the figures are a lower bound on the speedup over the reference implementation.

		cliff		spherical	
d	n	b=2	b=8	b=2	b=8
3	100	×5	×6	×2	×2
3	200	×3	×3	×2	×2
4	60	×14	×19	×6	×7
4	120	×16	×17	×8	×8
5	40	×5	×9	×6	×12
5	80	×7	×10	×9	×13
7	25	×3	×9	×3	×17
7	50	×3	×10	×5	×15
9	20	×1	×9	×2	×13
9	40	×2	×12	×3	×18
10	20	×1	×10	×2	×16
10	40	×1	×9	×2	×12

Table 3: Speedup of the C port over the fastest Python variant.

2.3 Recursion size

Node counts are reported by the C port; they measure the size of the recursion tree and are independent of language. Table 4 shows how the two datasets differ in the work they induce at equal n.

d	n	cliff	spherical
3	100	417	447
3	200	889	943
4	60	645	1 149
4	120	1 779	3 139
5	40	1 283	2 463
5	80	3 567	9 017
7	25	4 647	7 751
7	50	19 957	57 093
9	20	15 215	28 213
9	40	95 481	346 423
10	20	29 397	65 613
10	40	270 677	884 141

Table 4: Recursion nodes, C Section-2 with $b = 2$.

3 Agreement

Across 24 cases in which more than one variant completed, the largest relative spread between the hypervolumes returned by different variants was 1.1e-14, at $d = 10$, $n = 20$ on cliff (8 variants). Two independent algorithms (Section 2 and Section 4.2), two languages, and two base-case settings therefore agree to within floating-point rounding on every case measured. This is the strongest correctness statement available without an exact-arithmetic oracle, and it is what the reader should weigh most heavily: the timings below are only meaningful because all six variants compute the same number.

4 Discussion

The port is uniformly faster, by a fairly flat factor. With the base case matched to the Python default ($b = 2$), the C implementation runs between $\times 1$ and $\times 16$ faster than the fastest Python variant on the same input. The factor is notably stable across d and n, which is what one expects when the two programs execute the same recursion and differ only in interpretation overhead: this is a constant-factor win, not an algorithmic one.

The base case is the most valuable knob at high dimension. Raising the base case to $b = 8$ – stopping the recursion earlier and finishing by inclusion–exclusion – widens the gap to between $\times 2$ and $\times 19$. The benefit is negligible at $d = 3$ and largest at $d = 10$. The recursion pays a per-node cost that grows with d (simplification and cut selection both sweep all axes), while the base case pays 2^b box intersections regardless of depth; as d grows, trading nodes for a slightly more expensive base case becomes progressively more favourable. Anyone using this code at high dimension should tune b rather than accept the default.

Asymptotically faster is not yet actually faster. The Section-4.2 algorithm has the better bound, $O(n^{d/3} \operatorname{polylog} n)$, but on every case measured here it is between $\times 1.7$ and $\times 9.9$ *slower* than the Section-2 divide and conquer in the same language. Its constants are large: it manipulates symbolic step functions and terms where Section 2 manipulates boxes of doubles. The crossover, if it lies within reach at all, is beyond the problem sizes at which either implementation is usable in Python. The Section-4.2 code is best read as a faithful and

testable realisation of the paper's construction, not as the method of choice for computing hypervolumes today. Disabling its compression step changes little at these sizes, which is consistent with compression being a device for controlling asymptotic growth rather than a constant-factor optimisation.

The prefilter is free but useless here. Both datasets consist entirely of mutually non-dominated points by construction, so the $O(n^2 d)$ dominance filter can never remove anything. The measured difference between `prefilter=True` and `prefilter=False` is accordingly small and of either sign. This says nothing against the filter – on point sets with genuine dominance it is a large win – but it does mean these benchmarks isolate the recursion rather than the preprocessing.

Degenerate dominance structure is cheaper, not harder. At equal n and d, cliff induces a smaller recursion tree than spherical (Table 4) and correspondingly shorter times, even though both fronts are entirely non-dominated. Concentrating the order structure in two objectives leaves the remaining $d-2$ coordinates as noise that the simplification step can often collapse away, whereas a spherical front resists collapse in every axis at once. The intuition that "dominance confined to a low-dimensional slice" should be a hard case does not survive contact with the measurements.

Limits of these measurements. The following exceeded the 60.0 s limit and are shown as *t/o*: cliff at $d = 10$, $n = 20$ (Py YS); cliff at $d = 10$, $n = 40$ (Py §4.2 nc); cliff at $d = 10$, $n = 40$ (Py §4.2); cliff at $d = 9$, $n = 40$ (Py YS); spherical at $d = 10$, $n = 20$ (Py YS); spherical at $d = 10$, $n = 40$ (Py §4.2 nc); spherical at $d = 10$, $n = 40$ (Py §4.2); spherical at $d = 9$, $n = 40$ (Py YS); spherical at $d = 9$, $n = 40$ (Py §4.2 nc); spherical at $d = 9$, $n = 40$ (Py §4.2). Timings are single-run wall clock on one machine with no attempt to control turbo, thermal state, or background load; the reported factors should be read as order-of-magnitude, not as precise ratios. Node counts, by contrast, are exact and machine-independent.

5 Reproduction

```
make -C c bench_driver
python benchmarks/crossbench.py --timeout 120
python benchmarks/make_bench_tex.py
pdflatex paper/benchmarks.tex
```

Add `--quick` to `crossbench.py` for the smallest size per dimension only. All tables and the figures quoted in the discussion are generated from `benchmarks/results/crossbench.json`; this document is produced by `make_bench_tex.py` and should not be edited by hand.

Acknowledgements. The benchmark harness and this report were produced by the author, assisted by Claude Fable 5 (Anthropic), continuing the interactive agentic session that produced the implementations.